

操作系统课程设计，提高部分

Unix V6++ 系统的虚拟存储器（概要设计）

同济大学计算机系



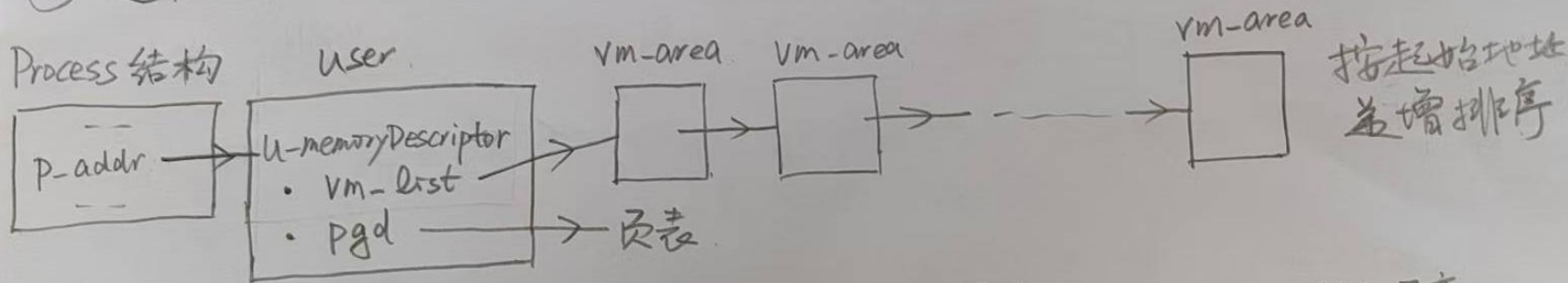
任务：为 Unix V6++ 系统添加虚拟存储器

- 要求：
 - exec执行时，为堆栈分配物理页框并装入命令行参数。
 - 其余逻辑页，缺页异常处理程序为其分配物理页框并予以初始化。
 - 进程释放物理页框时（比如 free、exit），注意维护哈希表和page数组。
- 核心数据结构：
 - 进程的内存描述符，管理虚地址空间，登记虚实地址映射关系
 - 系统的Page数组，登记每个物理页框的访问情况
 - 系统维护的一张哈希表，管理进程间需要共享的逻辑页

1. 关键数据结构

① 进程的内存描述符

vm-list 管理虚空间, pgd 是页目录或#用户页表。
每个逻辑段一个控制块 vm-area。



```
struct vm-area {
    v-start 虚空间起始地址
    v-length 虚空间长度
    prot 保护位 (u/s, R/w, P)
    f-start 文件中起始地址
    f-length 文件中长度
    *pgFault() 缺页异常处理程序
    next 指向下一个 vm-area
}
```

页表定义 UNIX V6++ 不变

```
struct pte {
    p: 1 在内存; 0 在交换区
    u/s: 略
    R/w: 略
    base: {
        物理页框号 P=1
        盘交换区地址 P=0
    }
    (盘交换区上的块号: 第1块是0, 每块4096字节)
}
```

page 1

② page 数组, 描述每个物理页框的使用状态

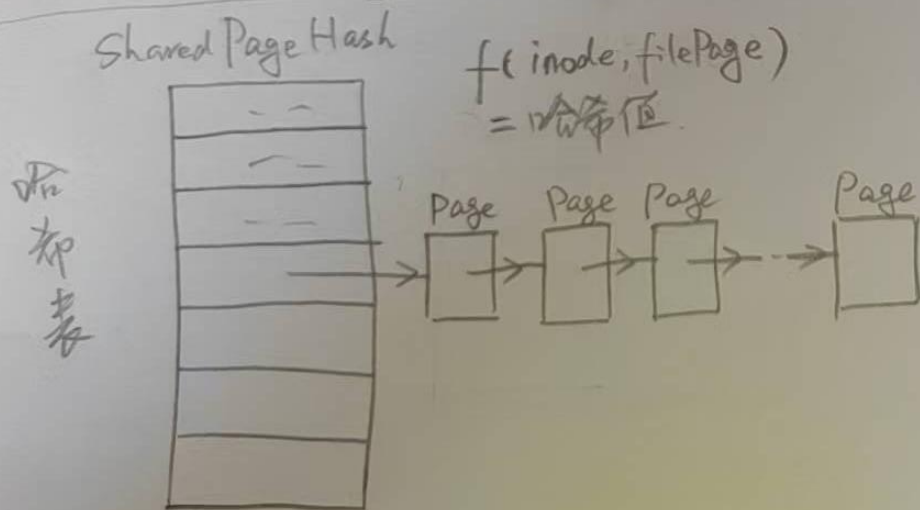
物理页框使用状态: 空闲 已分配

②.1 空闲物理页框列表



②.2 已分配物理页框, 按使用情况挂在不同的数据结构上.

- 物理页框
- ① 共享页: 可执行文件中的代码, 只读数据, 数据 (全局变量初值) 由 hash 表管理, 挂在这里 复制
 - ② 私有页: 堆, 栈, BSS, 进程的数据 (从可执行文件的数据段)
- ③ 页表, 页目录由 Process 管理. 由进程页表管理, 挂在这里



```

struct Page {
    inode (可执行文件 ID)
    filePage (文件中的偏移量)
    count 引用计数器
    pageNo 物理页框号
    next 哈希链指针
}
  
```

可以是文件中的数据块号, 4096 块一块。

page 2

③ 盘交换区管理

SwapPage 数组：盘交换区每一页一个控制块

```
struct swapPage {  
    ptr // 指针, 指向页表  
    page // 页号  
    swapPageNo // 盘交换区上页号  
    next // 空闲页表指针  
}
```

没设计完善, 大概率不需要
这三个字段

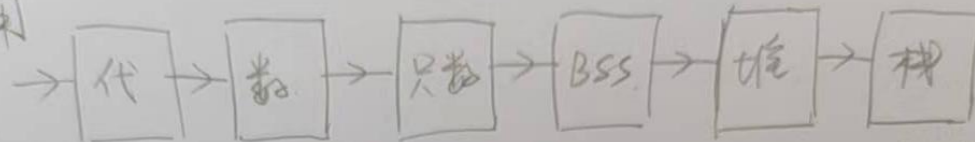
对我们的起步版本

page 3

2. exec 系统调用

——前不动

① 构造 vm-list



② 分配 1 个物理页框为用户栈，填入命令行参数， $page[-1].count++$ 。

③ 系统调用返回用户执行的应用程序。

3. fork 系统调用

① 复制父进程 Process, PPDA, vm-list
(user + 核心栈)

② 为子进程分配页表

③ 遍历 vm-list，对每个逻辑段，处理段内所有 PTE

父进程逻辑页，在哪个都可以，内存盘交换区随便。

Page 4

空？

RO？

RW？

子进程 PTE 清空

(这是父进程没访问过的 PTE)

复制父进程 PTE，

$Page[base].count++$;

父进程 PTE，RW 为 RO，复制给子进程

$Page[base].count++$;

后面有写

(若逻辑页在盘交换区，不cow缺页时，直接从磁盘调)

4. 缺页异常处理程序

功能: ① 识别非法内存访问 ② 扩展堆栈 ③ 请求调页 ④ 写时复制

① 识别非法内存访问

遍历 vm_list , 看 $cr2$ 落在哪个 vm_area , 记下来。如果没有匹配的 vm_area , 非法内存访问,

除非扩展堆栈, 否则是

② 堆栈扩展, 逻辑不变。

③ 请求调页 & 写时复制, 如下:

③.1 vm_area 是代码或只读数据, 从文件调页

• 计算虚地址为 $cr2$ 的指令在可执行文件中的偏移量。

$offset = f_start + (cr2 - v_start)$

• 计算 $offset$ 在文件中的页号 $page = offset \gg 12$

• Get 可执行文件的 $inode \#$: $inode$ • 用 $cr2$ 查页表, 定位映射所需 PTE

• 计算哈希值 $hash = sha(inode * page)$, 自定义。

• 遍历 $shared\ Page\ Hash[hash]$ 链表, 搜索 $inode$ 和 $filePage = page$ 的页

③.2 没找到: 分配物理页框 $frame$, 建立地址映射关系: $PTE.base = frame$, 填 $prot$, $flush$ 刷新 TLB

启动 IO 读入所缺页面 (read 函数, 参见 exec)

填写 $\text{Page}[\text{frame}]$, 将其插入哈希表 (哈希链表头部)
 $\text{count}++$ 别忘了

→ 找到, 即 (hash 命中) $\text{Page}[\text{frame}]$:

就用这个
页框

建立地址映射关系: $\text{PTE}.\text{base} = \text{frame}$, 填 prot , flush 刷新
 $\text{Page}[\text{frame}].\text{count}++$ TLB

3.2) vm-area 是数据段

像代码段那样, 计算 cr2 在文件中的页号 page 和页表中的 $\text{PTE}.$ frame1

• 分配
物理页框
frame

PTE 为空 (首次访问), 像代码段那样从文件调页, 分配物理页框
 读入磁盘文件中的全局变量初值, 把它挂在哈希树上 供该进程的所有
进程号查。

之后, 再分配一个物理页框 frame2 , 复制 frame1 的内容,
 frame2 挂进程页表, 建立地址映射关系.

• PTE 非空

RO 并且 $\text{P}==1$; 写时复制. (无 IO)

RO 但 $\text{P}==0$; 分配物理页框, 读入磁盘页 $\text{PTE}.\text{base}$
 (fork 时, 父进程的页已换出)

RW ; 同上

page 6

3.3 vm-area 是 bss 段。

- 首次访问 (不用 I/O)

注意: 页框刷。

注意维护哈希表

- 非首次访问

注意写时复制。

3.4 vm-area 是 堆和栈。

- 首次访问 (就是虚空中首次分配, 不用 I/O)

注意: 页框无需刷。

注意维护哈希表

- 非首次访问

注意写时复制。

page 7

3.5 栈扩展

3.6 堆扩展

} 首次访问. 同上. 注意需移动 vm-area
非首次访问.

5. 未尽的

进程终止 & 堆中释放动态内存。

注意维护 page 数组。

page 8